

Algoritmo t-SNE

El algoritmo t-SNE (t-Distributed Stochastic Neighbor Embedding) es un algoritmo diseñado específicamente para la visualización de datos de alta dimensionalidad.

A diferencia de algoritmos como PCA (Análisis de Componentes Principales), que buscan preservar la varianza global y las grandes distancias del conjunto de datos, t-SNE es una técnica no lineal que se enfoca en preservar la estructura local. Esto significa que si dos puntos están muy cerca en el espacio de alta dimensión (por ejemplo, tienen características muy similares), t-SNE hará todo lo posible por mantenerlos juntos en el mapa de baja dimensión (2D o 3D).

1. Descubrimiento de Clústers Nativos (Estructuras No Lineales)

Cuando los datos tienen relaciones complejas y los grupos no pueden separarse mediante líneas o planos rectos (hiperplanos), las técnicas lineales fallan. t-SNE mapea estas topologías en "manifolds" (variedades) de baja dimensión, revelando agrupaciones o patrones que de otro modo estarían ocultos.

2. Análisis de Embeddings en Machine Learning y NLP

En modelos de aprendizaje profundo (Deep Learning), las capas intermedias y finales generan vectores de características (embeddings) densos y de alta dimensionalidad.

Procesamiento de Lenguaje Natural (NLP): Para visualizar embeddings de palabras (como Word2Vec o vectores de transformers). Permite ver cómo el algoritmo agrupa por vecindad semántica palabras con significados o contextos similares (ej. países juntos, verbos en pasado juntos).

Visión por Computadora: Para analizar las representaciones internas de una Red Neuronal Convolutiva (CNN). Permite verificar si la red está separando correctamente las clases antes de la capa de clasificación final.

Limitaciones clave a tener en cuenta en estos problemas:

- No sirve para reducir dimensiones como paso previo a otro algoritmo de Machine Learning: t-SNE no genera una función de mapeo reutilizable. Si llegan nuevos datos, no puedes "transformarlos" directamente; tendrías que volver a correr todo el algoritmo.
- Las distancias entre clústers no son significativas: t-SNE preserva la estructura local, no la global. Si el clúster A está el doble de lejos del clúster B que del C en el gráfico, no significa necesariamente que sea el doble de diferente en la realidad.

Aplicación del Algoritmo t-SNE en el proyecto

En SMC, el mercado se mueve por la acumulación y distribución de órdenes institucionales en zonas específicas. t-SNE te permite tomar docenas de métricas numéricas de la microestructura del mercado y proyectarlas en un mapa 2D para ver si los conceptos teóricos de SMC realmente forman firmas o clústers matemáticos consistentes.

El algoritmo t-SNE sirve de guía el entrenamiento de GMM (Gaussian Mixture Models) sin mezclar las dimensiones. Debemos visualizar a t-SNE como un auditor o cartógrafo y a GMM/EM como el constructor de fronteras. Dado que t-SNE no puede procesar datos nuevos en vivo, su trabajo ocurre exclusivamente en la fase de diseño y entrenamiento (offline).

t-SNE guía a GMM resolviendo los tres grandes problemas del algoritmo Expectation-Maximization en alta dimensión: la inicialización a ciegas, el sobreajuste por ruido y la selección del número óptimo de componentes (K).

A continuación se detalla el proceso de cómo t-SNE actúa como el lazarillo matemático de tu GMM:



Fase de Investigación: El Rol de t-SNE como Supervisor de Topología para GMM

En espacios de alta dimensionalidad (como nuestra matriz de variables de Smart Money Concepts de 20 a 45 columnas), entrenar un **Modelo de Mezclas Gaussianas (GMM)** directamente mediante el algoritmo **Expectation-Maximization (EM)** es un peligro estadístico. Debido a la *maldición de la dimensionalidad*, las densidades de probabilidad en $45D$ se vuelven difusas, provocando que EM converja en máximos locales sin sentido operativo, mezclando ruido minorista con inyecciones de capital institucional.

t-SNE (t-Distributed Stochastic Neighbor Embedding) no es un algoritmo predictivo, pero actúa como nuestro **auditor y mapa topológico**. Su rol no es operar en vivo, sino guiar y validar de forma estricta el proceso de entrenamiento del GMM en el espacio de alta dimensión.

La simbiosis matemática funciona bajo la premisa de **transferencia de etiquetas topológicas** desde un espacio de baja dimensión (2D) hacia las densidades del espacio original de alta dimensión (45D).

Protocolo de Entrenamiento Guiado por t-SNE

El proceso de entrenamiento no es directo; sigue un flujo secuencial donde la geometría no lineal descubierta por t-SNE dicta cómo el algoritmo EM debe ajustar las campanas de Gauss en la base de datos real:

1. Proyección y Filtrado No Lineal (t-SNE 2D)

- Tomamos la matriz histórica de eventos SMC estandarizada/normalizada de $N \times 45$ variables microestructurales de nuestras zonas SMC (Order Blocks, FVGs, Liquidity Sweeps).
- Corremos el algoritmo de t-SNE para colapsar las características a una matriz de coordenadas cartesianas de $N \times 2$ (`tSNE_X0`, `tSNE_X1`).
- Al graficar este plano y colorear los puntos según su desenlace real en el mercado (ganadoras/a favor de la tendencia externa/principal `1` vs perdedoras/en contra de la tendencia externa/principal `0`), t-SNE nos revelará la verdad matemática: **si las firmas de volumen institucional forman "islas" aisladas de las trampas minoristas**.

2. Extracción de Índices Geométricos (Clustering Supervisor)

- Aplicamos el algoritmo GMM bidimensional **únicamente sobre el mapa 2D generado por t-SNE**.
- Identificamos el ID de la "isla" o clúster que concentra la mayor cantidad de zonas SMC ganadoras históricas (el clúster de alta probabilidad institucional).
- Guardamos los índices de las filas (los `Eventos_ID`) que pertenecen estrictamente a esa isla ganadora.

3. Ajuste Restringido vía EM en Alta Dimensión (GMM 45D)

- En lugar de pedirle al algoritmo Expectation-Maximization que adivine la forma de todo el mercado en 45D, **restringimos su inicialización**.
- Aislamos en la matriz original de 45 dimensiones *únicamente* los vectores correspondientes a los índices guardados de la isla ganadora de t-SNE.
- Ejecutamos el algoritmo EM para calcular la media (μ) y la matriz de covarianza (Σ) de ese componente específico en $45D$.

 **El resultado:** Obligamos a EM a moldear una hiper-elipsoide matemática en 45D que encapsula perfectamente una firma que t-SNE ya demostró visualmente que es real, limpia de ruido y geoméricamente consistente.

Pipeline de Acoplamiento de Datos

El siguiente esquema resume cómo t-SNE actúa como el plano arquitectónico que guía la construcción de las densidades probabilísticas en el GMM final:





[Algoritmo EM / GMM en Alta Dimensión] → Estima parámetros óptimos (μ , Σ) para Inferencia en Vivo

¿Por qué este enfoque guiado es superior?

1. **Estabilidad en el algoritmo EM:** Al inicializar las Gaussianas con datos previamente filtrados topológicamente por t-SNE, evitamos que las matrices de covarianza del GMM se vuelvan singulares o colapsen numéricamente.
2. **Defensa ante falsos positivos:** Si t-SNE muestra que en un determinado régimen macroeconómico los puntos de pérdidas y ganancias están entrelazados como una masa caótica en 2D, el sistema descarta esa sección de datos, impidiendo que el GMM intente ajustar un modelo predictivo sobre puro ruido.

Aplicar t-SNE para modelar la **estructura interna atemporal** de los eventos de Smart Money Concepts (SMC) es una estrategia excelente para dar el salto del análisis visual discrecional (dibujar zonas a mano en el gráfico) al reconocimiento algorítmico de patrones cuantitativos. La **estructura externa temporal** se modelará mediante Modelos Ocultos de Markov (HMM).

```
In [6]: package TSNE{

    use strict;
    use warnings;
    use List::Util qw(max);
    use Data::Dump qw(dump);
    use AI::MXNet qw(mx nd);
    use Time::HiRes qw(time);
    use POSIX qw(ceil);

    our $MACHINE_EPSILON = 2.220446049250313e-16;

    sub new {
        my ($class, %args) = @_;

        my $self = {
            n_components          => $args{n_components} // 2,
            perplexity            => $args{perplexity} // 30.0,
            early_exaggeration    => $args{early_exaggeration} // 12.0,
            learning_rate         => $args{learning_rate} // 'auto',
            n_iter                => $args{max_iter} // $args{n_iter} // 1000,
            n_iter_without_progress => $args{n_iter_without_progress} // 300,
            n_iter_early_exaggeration => $args{n_iter_early_exaggeration} // 250,
            min_grad_norm        => $args{min_grad_norm} // 1e-7,
            metric               => $args{metric} // 'euclidean',
            init                 => $args{init} // 'random',
            verbose              => $args{verbose} // 0,
            random_state         => $args{random_state},
            method               => $args{method} // 'exact',
            n_iter_check         => $args{n_iter_check} // 25,
            embedding_           => undef
        };

        return bless($self, $class);
    }

    # =====
    # Pairwise squared euclidean distances for input data X: 'sqeuclidean' metrics
    # scipy/spatial/distance.py
    # =====
    sub pdist {
        my ($self, $X) = @_;

        if (ref($X) =~ /^AI::MXNet::NDArray(?:::Slice)?$/){
            $X = $X->astype('float64');
            my $n_samples = $X->shape->[0];

            # ||x_i - x_j||^2 = x_i^2 - 2*x_i*x_j^T + x_j^2 (Matriz completa intermedia)
            my $X_sum_squares = nd->sum($X * $X, axis => 1, keepdims => 1);
            my $X_inner_prod = nd->dot($X, $X->T);
            my $sqdistances = $X_sum_squares - (2 * $X_inner_prod) + $X_sum_squares->T;
        }
    }
}
```

```

$sqdistances = nd->clip($sqdistances, a_min => 0.0, a_max => 'Inf');

# 1. Obtener los índices del triángulo superior estricto (offset k=1 para omitir diagonal)
# 2. Devuelve condensado al recolectar de forma puramente vectorial usando los índices est
return nd->gather_nd($sqdistances, nd->triu_indices($n_samples, 1));
}elseif(ref($X) eq 'ARRAY'){
# X es un arreglo bidimensional [[x1, y1], [x2, y2], ...]
my $n_samples = scalar @$X;
my $n_components = scalar @{$X->[0]};

my @condensed;

# Recorremos el triángulo superior estricto usando dos índices (i < j)
for my $i (0 .. $n_samples - 2) {
    my $row_i = $X->[$i];

    for my $j ($i + 1 .. $n_samples - 1) {
        my $row_j = $X->[$j];

        # Calcular la distancia euclidiana al cuadrado entre la muestra i y la muestra j
        my $sq_dist = 0;
        for my $k (0 .. $n_components - 1) {
            my $diff = $row_i->[$k] - $row_j->[$k];
            $sq_dist += $diff * $diff;
        }

        # Control de estabilidad numérica (clip a mínimo 0.0)
        $sq_dist = 0.0 if $sq_dist < 0.0;

        push @condensed, $sq_dist;
    }
}

return \@condensed; # Retorna una referencia al arreglo plano indexado
}

# =====
# La implementación de squareform está perfectamente vectorizado en C++,
# pero la ralentización no ocurría por procesar datos en Perl, sino por el costo de crear,
# destruir y transponer tensores intermedios en cada iteración.
# Al eliminar squareform y mantener la matriz fija en un tamaño constante de 5x5,
# elimina ese flujo constante de creación de memoria e instrucciones en el motor de C++.
# scipy/spatial/distance.py
# =====
sub squareform {
    my ($self, $X) = @_;
    my ($n_rows, $n_cols) = @{$X->shape};

    # CASO 1: Vector plano unidimensional -> Convertir a Matriz Cuadrada Simétrica
    if ($X->ndim == 1) {
        my $M = $n_rows;

        # Resolver la ecuación de SciPy para hallar N:  $M = N * (N - 1) / 2$ 
        #  $N^2 - N - 2M = 0 \rightarrow N = (1 + \sqrt{1 + 8M}) / 2$ 
        my $n_samples = int((1 + sqrt(1 + 8 * $M)) / 2);

        # 1. Obtener los índices de destino para el triángulo superior estricto
        my $indices_upper = nd->triu_indices($n_samples, 1);

        # 2. Inicializar la matriz cuadrada vacía
        my $matrix = nd->zeros([$n_samples, $n_samples], dtype => $X->dtype);

        # 3. Dispersar los valores del vector en el triángulo superior
        $matrix = nd->scatter_nd(data => $X, indices => $indices_upper, shape => [$n_samples, $n_

        # 4. Sometemos la matriz a simetrización completa (Reflejar triángulo superior en el infe
        return $matrix + $matrix->T;
    }

    # CASO 2: Matriz cuadrada bidimensional -> Convertir a Vector Condensado
    elsif ($X->ndim == 2) {
        if ($n_rows != $n_cols) {
            die "Error in squareform: The matrix argument must be square.";
        }
    }
}

```

```

# 1. Obtener los índices del triángulo superior estricto (offset k=1 para omitir diagonal)
# 2. Devuelve condensed al recolectar de forma puramente vectorial usando los índices est
return nd->gather_nd($X, nd->triu_indices($n_rows, 1));
}
else {
    die sprintf("Error in squareform: The first argument must be one or two dimensional. Arra
}
}

# Binary search for sigmas of conditional Gaussians.
# sklearn/manifold/_utils.pyx
# This approximation reduces the computational complexity from  $O(N^2)$  to
#  $O(uN)$ .
#
# Parameters
#-----
# sqdistances : ndarray of shape (n_samples, n_neighbors), dtype=np.float32
#     Distances between training samples and their k nearest neighbors.
#     When using the exact method, this is a square (n_samples, n_samples)
#     distance matrix. The TSNE default metric is "euclidean" which is
#     interpreted as squared euclidean distance.
#
# desired_perplexity : float
#     Desired perplexity ( $2^{\text{entropy}}$ ) of the conditional Gaussians.
#
# verbose : int
#     Verbosity level.
#
# Returns
#-----
# P : ndarray of shape (n_samples, n_neighbors), dtype=np.float64
#     Probabilities of conditional Gaussian distributions  $p_{i|j}$ .

# Fully Vectorized _binary_search_perplexity
# sklearn/manifold/_utils.pyx

sub _binary_search_perplexity_scalar {
    my ($self, $sqdistances, $desired_perplexity, $verbose) = @_;

    # Forzar float32 en la entrada para que coincida con sklearn
    # $sqdistances = $sqdistances->astype('float32');
    my ($n_samples, $n_neighbors) = @{$sqdistances->shape};
    my $using_neighbors = $n_neighbors < $n_samples;

    my $n_steps = 100;
    my $EPSILON_DBL = 1e-8;
    my $PERPLEXITY_TOLERANCE = 1e-5;
    my $desired_entropy = log($desired_perplexity);

    # Matriz final P en float64
    my $P = nd->zeros([$n_samples, $n_neighbors], dtype => 'float64', ctx => $sqdistances->cont
    my $beta_sum = 0.0;

    for my $i (0 .. $n_samples-1) {
        my $beta = 1.0;
        my $beta_min = -9e999; # -inf
        my $beta_max = 9e999; # +inf

        # Extraemos la fila actual a un array nativo de Perl para evitar
        # el ruido de precisión de los operadores de MXNet
        my $row_data = $sqdistances->slice($i)->asarray();

        # Array temporal de Perl para la fila  $P_i$  actual
        my $Pi_row = [(0.0) x $n_neighbors];

        for (my $step = 0; $step < $n_steps; $step++) {
            my $sum_Pi = 0.0;

            # 1. Calcular Exponenciales y acumular la suma secuencialmente (Idéntico a Cython)
            for my $j (0 .. $n_neighbors-1) {
                if ($j != $i || $using_neighbors) {
                    $Pi_row->[$j] = exp(-$row_data->[$j] * $beta);
                    $sum_Pi += $Pi_row->[$j];
                } else {

```

```

        $Pi_row->[$j] = 0.0;
    }
}

$sum_Pi = $EPSILON_DBL if $sum_Pi == 0.0;

# 2. Normalizar y calcular sum_disti_Pi en el mismo orden secuencial
my $sum_disti_Pi = 0.0;
for my $j (0 .. $n_neighbors-1) {
    $Pi_row->[$j] /= $sum_Pi;
    $sum_disti_Pi += $row_data->[$j] * $Pi_row->[$j];
}

my $entropy = log($sum_Pi) + $beta * $sum_disti_Pi;
my $entropy_diff = $entropy - $desired_entropy;

# 3. Guardar la fila calculada en la matriz P de MXNet
# Lo hacemos aquí para asegurar que conserve el último estado de la búsqueda
my $Pi_nd = nd->array($Pi_row, dtype => 'float64', ctx => $sqdistances->context);
$P->slice($i, ':')->set($Pi_nd);

# Criterio de parada
last if abs($entropy_diff) <= $PERPLEXITY_TOLERANCE;

# Actualización de Beta (Búsqueda Binaria)
if ($entropy_diff > 0.0) {
    $beta_min = $beta;
    $beta = $beta_max > 1e300 ? $beta * 2.0 : ($beta + $beta_max) / 2.0;
} else {
    $beta_max = $beta;
    $beta = $beta_min < -1e300 ? $beta / 2.0 : ($beta + $beta_min) / 2.0;
}
}
$beta_sum += $beta;

if ($verbose && (((i + 1) % 1000) == 0 || (i + 1) == $n_samples)) {
    print "[t-SNE] Computed conditional probabilities for sample ", $i + 1, " / ", $n_sampl
}
}

if ($verbose) {
    printf "[t-SNE] Mean sigma: %f\n", sqrt($n_samples / $beta_sum);
}

return $P;
}

# Fully Vectorized _binary_search_perplexity
# sklearn/manifold/_utils.pyx
sub _binary_search_perplexity_mxnet {
    my ($self, $sqdistances, $desired_perplexity, $verbose, $precomputed_beta) = @_;

    my ($n_samples, $n_neighbors) = @{$sqdistances->shape};
    my $sqdistances_f64 = $sqdistances->astype('float64');

    # SI YA TENEMOS BETA, EVALUAMOS DIRECTAMENTE (Ahorro masivo de cálculos)
    if (defined $precomputed_beta) {
        my $exponent = -$sqdistances_f64 * $precomputed_beta;
        my $floor_tensor = nd->ones($exponent->shape, dtype => 'float64') * -708.3964185322641;
        $exponent = $exponent->maximum($floor_tensor);
        my $row_P = nd->exp($exponent);

        # Si es matriz cuadrada (entrenamiento), aplicar máscara diagonal
        if ($n_neighbors == $n_samples) {
            my $diag_mask = nd->ones([$n_samples, $n_neighbors], dtype => 'float64') - nd->eye($n
            $row_P = $row_P * $diag_mask;
        }

        my $sum_Pi = nd->sum($row_P, axis => 1, keepdims => 1)->maximum(1e-8);
        return $row_P / $sum_Pi;
    }

    my $n_steps = 100;
    my $EPSILON_DBL = 1e-8;
    my $PERPLEXITY_TOLERANCE = 1e-5;

```

```

my $desired_entropy = log($desired_perplexity);

# Parámetros internos en float64 (equivalente a cdef double beta)
my $beta = nd->ones([$n_samples, 1], dtype => 'float64');
my $beta_min = nd->ones([$n_samples, 1], dtype => 'float64') * -1e20;
my $beta_max = nd->ones([$n_samples, 1], dtype => 'float64') * 1e20;

my $diag_mask;
if ($n_neighbors == $n_samples) {
    $diag_mask = nd->ones([$n_samples, $n_neighbors], dtype => 'float64') - nd->eye($n_sample
}

my $row_P_normalized;

# Global binary search loop
for my $step (0 .. $n_steps - 1) {

    # === AJUSTE DE SUBFLUJO: Prevenir que MXNet trunque prematuramente a 0.0 ===
    my $exponent = -$sqdistances_f64 * $beta;

    # -708.39 es el límite inferior exacto para float64 donde math.exp se vuelve 0.0
    # Usamos el método .maximum() de MXNet para asegurar consistencia de dtypes
    my $floor_tensor = nd->ones($exponent->shape, dtype => 'float64') * -708.3964185322641;
    $exponent = $exponent->maximum($floor_tensor);

    my $row_P = nd->exp($exponent);

    if (defined $diag_mask) {
        $row_P = $row_P * $diag_mask; # float64 * float64
    }

    # 2. Compute sum across rows
    my $sum_Pi = nd->sum($row_P, axis => 1, keepdims => 1)->maximum($EPSILON_DBL);

    # 3. Normalize (Se mantiene en float64)
    $row_P_normalized = $row_P / $sum_Pi;

    # 4. Compute sum_disti_Pi (Operar sqdistances_f64 para mantener homogeneidad con row_P_no
    my $sum_disti_Pi = nd->sum($sqdistances_f64 * $row_P_normalized, axis => 1, keepdims => 1

    # 5. Compute Entropy globally
    my $entropy = nd->log($sum_Pi) + $beta * $sum_disti_Pi;
    my $entropy_diff = $entropy - $desired_entropy;

    my $is_greater = ($entropy_diff > 0.0)->astype('float64');
    my $is_less = nd->ones([$n_samples, 1], dtype => 'float64') - $is_greater;

    my $mask_max_init = ($beta_max >= 1e19)->astype('float64');
    my $mask_min_init = ($beta_min <= -1e19)->astype('float64');

    # Actualizaciones de límites estables
    $beta_min = ($is_greater * $beta) + ($is_less * $beta_min);
    $beta_max = ($is_less * $beta) + ($is_greater * $beta_max);

    # Posibles caminos matemáticos (Todos en float64 inherente)
    my $b_greater_max_init = $beta * 2.0;
    my $b_greater_not_init = ($beta + $beta_max) / 2.0;
    my $b_less_min_init = $beta / 2.0;
    my $b_less_not_init = ($beta + $beta_min) / 2.0;

    # Construcción limpia del vector de Betas usando álgebra de conmutación 100% float64
    $beta = ($is_greater * $mask_max_init * $b_greater_max_init) +
        ($is_greater * (nd->ones([$n_samples, 1], dtype => 'float64') - $mask_max_init) *
        ($is_less * $mask_min_init * $b_less_min_init) +
        ($is_less * (nd->ones([$n_samples, 1], dtype => 'float64') - $mask_min_init) * $b

}

# Saves beta to avoid recalculation when predicting
# Cuando introduces un nuevo punto $m$ (fuera de la muestra), necesitas calcular la afinida
# Para ello, openTSNE define dos enfoques que puedes optimizar si guardaste los tensores:
# -> Uso de Varianzas Locales de los Vecinos (Enfoque openTSNE):
#     En lugar de correr una nueva búsqueda binaria de 100 pasos para hallar una $b_m$ p
#     se buscan sus $k$ vecinos más cercanos en el espacio original y se asigna como su $b_e
# -> Reutilizar la Matriz de Distancias Inversas:
#     Te saltas la costosa calibración de entropía y pasas directo a evaluar la exponencial.

```

```

$self->{beta_train_} = $beta if ref($self) eq 'TSNE';

# Retorna P_normalized conservando su estructura float64 final
return $row_P_normalized;
}

sub _joint_probabilities {
    my ($self, $distances, $desired_perplexity, $verbose) = @_;

    # Expandir el vector a matriz cuadrada para que $self->_binary_search_perplexity pueda operar
    $distances = $distances->astype('float32');
    # 1. Obtiene probabilidades condicionales globales en formato matricial
    my $conditional_P = $self->_binary_search_perplexity_mxnet($distances, $desired_perplexity,

    # 2. Symmetrize:  $P = (conditional\_P + conditional\_P.T)$ 
    my $P = $conditional_P + $conditional_P->T;

    # 3. Retorna el vector condensado P con suelo de seguridad máquina epsilon
    return nd->maximum_scalar($self->squareform($P) / nd->maximum_scalar(nd->sum($P), $MACHINE_EPSILON);
}

sub _kl_divergence {
    my ($self, $params, $obj_args_ref, $skip_num_points, $compute_error) = @_;
    my ($P, $degrees_of_freedom, $n_samples, $n_components) = @$obj_args_ref;

    my $X_embedded = $params->reshape([$n_samples, $n_components]);

    # 1. Distancias euclidianas al cuadrado
    my $dist = $self->pdist($X_embedded, "sqeuclidean");

    # 2. Distribución de Student-t
    $dist /= $degrees_of_freedom;
    $dist += 1.0;
    $dist **= ($degrees_of_freedom + 1.0) / -2.0;

    # 3. Cálculo de Q (Calculando la suma nativamente en MXNet para mantener precisión)
    my $sum_dist = nd->sum($dist)->asscalar;
    my $Q = nd->maximum_scalar($dist / (2.0 * $sum_dist), scalar => $MACHINE_EPSILON);

    # 4. Divergencia KL (Función Objetivo)
    my $kl_divergence;
    if ($compute_error) {
        my $P_clamped = nd->maximum_scalar($P, scalar => $MACHINE_EPSILON);
        $kl_divergence = 2.0 * nd->dot($P, nd->log($P_clamped / $Q))->asscalar;
    } else {
        $kl_divergence = 'Nan';
    }

    # 5. Gradiente
    my $PQd = $self->squareform(($P - $Q) * $dist);

    # Inicializar la matriz de gradientes con ceros usando el mismo tipo de datos
    my $grad;

    # Replicamos el bucle exacto respetando skip_num_points
    if ($skip_num_points == 0){
        $grad = (($PQd->sum(axis => 1, keepdims => 1) * $X_embedded) - nd->dot($PQd, $X_embedded))
    } else {
        for my $i ($skip_num_points .. $n_samples - 1) {
            $grad = nd->zeros([$n_samples, $n_components], dtype => $params->dtype, ctx => $params->ctx);
            # Asignar np.dot(PQd[i], X_embedded[i] - X_embedded) a la fila correspondiente del grad
            $grad->slice($i, ':')->set(nd->dot($PQd->slice($i), $X_embedded->slice($i) - $X_embedded));
        }
    }

    # Escalamiento final del gradiente
    $grad *= 2.0 * ($degrees_of_freedom + 1.0) / $degrees_of_freedom;

    return ($kl_divergence, $grad->reshape([-1]));
}

# Reference: _gradient_descent from sklearn/manifold/_t_sne.py
sub _gradient_descent {
    my ($self, $objective, $p0, %args) = (splice(@_, 0, 3), @_);

```



```

my $it = $args{it};
my $n_iter = $args{n_iter};
my $objective_args = $args{objective_args} // [];
my $skip_num_points = $args{skip_num_points} // 0;
my $n_iter_check = $args{n_iter_check} // $self->{n_iter_check} // 1;
my $learning_rate = $args{learning_rate};
my $momentum = $args{momentum} // 0.8;
my $min_gain = $args{min_gain} // 0.01;
my $min_grad_norm = $args{min_grad_norm} // 1e-7;
my $verbose = $args{verbose} // 0;
my $n_iter_without_progress = $args{n_iter_without_progress} // 300;

my ($error, $grad, $i);
my $p = $p0->copy->reshape([-1]);
my $update = nd->zeros_like($p);
my $gains = nd->ones_like($p);
my $best_error = nd->inf;
my $best_iter = $it;

my $tic = time();
for $i ($it .. $n_iter - 1) {
    my $check_convergence = (($i + 1) % $n_iter_check == 0);
    my $compute_error = ($check_convergence || $i == $n_iter - 1);

    # --- Medir la función objetivo (_kl_divergence) ---
    ($error, $grad) = $objective->('TSNE', $p, $objective_args, $skip_num_points, $compute_error);

    my $inc = nd->lesser_scalar(($update * $grad), 0.0); # 1. Dirección del gradiente
    $gains = nd->where($inc, $gains + 0.2, $gains * 0.8); # 2. Ganancias adaptativas
    $gains = nd->clip($gains, a_min => $min_gain, a_max => 'Inf');
    $grad *= $gains; # 3. Actualizar gradiente
    $update = ($momentum * $update) - ($learning_rate * $grad); # 4. Actualizar Momentum
    $p += $update; # 5. Actualizar Posición

    # 6. Evaluación de Criterios de Parada e Impresión (Ocurre estrictamente bajo check_convergence)
    if ($check_convergence) {
        my $grad_norm = nd->norm($grad)->asscalar;

        if ($verbose && $verbose >= 2) {
            nd->printf("[t-SNE] Iteration %d : error = %.8f, gradient norm = %.8f (%s iterations\n", $i + 1, $error, $grad_norm, $i + 1 - $best_iter);
        }

        if ($error < $best_error) {
            $best_error = $error;
            $best_iter = $i;
        } elsif ($i - $best_iter > $n_iter_without_progress) {
            if ($verbose && $verbose > 0) {
                printf "[t-SNE] Iteration %d: did not make any progress during the last %d episodes\n", $i + 1, $n_iter_without_progress;
            }
            last;
        }

        if ($grad_norm <= $min_grad_norm) {
            if ($verbose && $verbose > 0) {
                nd->printf("[t-SNE] Iteration %d: gradient norm %.8f. Finished.\n", $i + 1, $grad_norm);
            }
            last;
        }
    }
}

return ($p, $best_error, $best_iter);
}

sub _fit {
    my ($self, $X, %args) = (splice(@_, 0, 2), skip_num_points=> 0, @_);
    # Private function to fit the model using X as training data.

    my ($n_samples, $X_embedded) = $X->len;

    # Manejo dinámico de Learning Rate Automático
    if ($self->{learning_rate} eq 'auto') {
        $self->{learning_rate} = max($n_samples / $self->{early_exaggeration} / 4.0, 50);
    } else {

```

```

    $self->{_learning_rate} = $self->{learning_rate} // 200;
}

my $P;
if ($self->{method} eq 'exact'){
    my $distances;
    if ($self->{metric} eq 'precomputed'){
        $distances = $X;
    }else{
        if ($self->{verbose}) {
            print "[t-SNE] Computing pairwise distances...\n";
        }
        if ($self->{metric} eq 'euclidean'){
            $distances = $self->squareform($self->pdist($X));
        } # else #
    }
}

# compute the joint probability distribution for the input space
$P = $self->_joint_probabilities($distances, $self->{perplexity}, $self->{verbose});

die "All probabilities should be finite"
    unless nd->all(nd->isfinite($P))->asscalar;

die "All probabilities should be non-negative"
    unless nd->all($P >= 0)->asscalar;

die "All probabilities should be less than or equal to one"
    unless nd->all($P <= 1)->asscalar;
} # else knn

# Preparacion del p0
if (ref($self->{init}) && ref($self->{init}) =~ /^AI::MXNet::NDArray(?:::Slice)?$/) {
    $X_embedded = $self->{init};
} elsif (!ref($self->{init}) && $self->{init} eq 'pca') {
    die "'init' as 'pca', is not implemented.\n";
} elsif (!ref($self->{init}) && $self->{init} eq 'random') {
    # Inicialización aleatoria por defecto
    mx->random->seed($self->{random_state}) if defined $self->{random_state};
    $X_embedded = nd->random->normal(loc => 0.0, scale => 1e-4, shape => [$n_samples * $self->{n_components}]);
} else {
    die "'init' must be 'pca', 'random', or a numpy array.\n";
}

my $degrees_of_freedom = max($self->{n_components} - 1, 1);

#===Runs t-SNE.===#
# t-SNE minimizes the Kullback-Leiber divergence of the Gaussians P
# and the Student's t-distributions Q. The optimization algorithm that
# we use is batch gradient descent with two stages:
# * initial optimization with early exaggeration and momentum at 0.5
# * final optimization with momentum at 0.8

my $params = $X_embedded->reshape([-1]);

# Step 4: Phase 1 - Early Exaggeration Optimization
if ($self->{verbose}) {
    print "[t-SNE] Starting Early Exaggeration Phase...\n";
}

# Learning schedule (part 1): do 250 iteration with lower momentum but
# higher learning rate controlled via the early exaggeration parameter

my %opt_args = (it => 0,
    n_iter_check      => $self->{n_iter_check},
    min_grad_norm     => $self->{min_grad_norm},
    _learning_rate    => $self->{_learning_rate},
    verbose           => $self->{verbose},
    skip_num_points   => $args{skip_num_points},
    objective_args    => [$P, $degrees_of_freedom, $n_samples, $self->{n_components}],
    n_iter_without_progress => $self->{n_iter_early_exaggeration}, # Limit of i
    n_iter            => $self->{n_iter},
    momentum          => 0.5,
);

```

```

my $tic = time();
$P *= $self->{early_exaggeration};
my $obj_func = \&{'TSNE::_kl_divergence'};
($params, my $kl_divergence, my $it) = $self->_gradient_descent($obj_func, $X_embedded, %op

# Step 5: Phase 2 - Final Optimization (Without Exaggeration)
if ($self->{verbose}) {
    printf "[t-SNE] KL divergence after %d iterations with early exaggeration: %.8f in %0.3fs\n", $it, $kl_divergence, $tic - $t0;
}

$P /= $self->{early_exaggeration};

my $remaining = $self->{n_iter} - $self->{n_iter_early_exaggeration};
if ($it < $self->{n_iter_early_exaggeration} || $remaining > 0){
    $opt_args{n_iter} = $self->{n_iter};
    $opt_args{it} = $it + 1;
    $opt_args{momentum} = 0.8;
    $opt_args{n_iter_without_progress} = $self->{n_iter_without_progress};
    ($params, $kl_divergence, $it) = $self->_gradient_descent($obj_func, $params, %opt_args)
}

# Save the final number of iterations
$self->{n_iter_} = $it;

if ($self->{verbose}) {
    printf "[t-SNE] KL divergence after %d iterations: %.8f in %0.3fs\n", $it + 1, $kl_divergence, $tic - $t0;
}

$X_embedded = $params->reshape([n_samples, $self->{n_components}]);
$self->{kl_divergence_} = $kl_divergence;

return $X_embedded;
}

sub _check_params_vs_input {
    my ($self, $X) = @_;
    my $n_samples = $X->shape->[0];

    # Si la perplexity está configurada en 'auto', openTSNE suele estimarla (ej. N / 3)
    if (defined $self->{perplexity} && $self->{perplexity} eq 'auto') {
        $self->{perplexity} = max(1, ceil($n_samples / 3));
        if ($self->{verbose}) {
            print "[t-SNE] Perplexity automáticamente estimada en: ", $self->{perplexity}, "\n";
        }
    }

    # Validación quirúrgica estricta estilo openTSNE / Sklearn
    if ($self->{perplexity} >= $n_samples) {
        die "Error: Perplexity must be less than n_samples. n_samples = $n_samples, perplexity = $self->{perplexity}";
    }

    if ($n_samples < 3 * $self->{perplexity} && $self->{verbose}) {
        warn "[t-SNE] Warning: n_samples ($n_samples) es menor que 3 * perplexity. El resultado puede ser de baja calidad";
    }
}

sub fit_transform {
    my ($self, $X) = @_;

    #Fit X into an embedded space and return that transformed output.
    #
    #Parameters
    #-----
    #X : ndarray of shape (n_samples, n_features) or (n_samples, n_samples)
    #    If the metric is 'precomputed' X must be a square distance
    #    matrix. Otherwise it contains a sample per row. If the method
    #    is 'exact', X may be a sparse matrix of type 'csr', 'csc'
    #    or 'coo'. The method is 'barnes_hut' with its metric
    #    'precomputed' where X may be a precomputed sparse graph is not implemented.
    #
    #Returns
    #-----
    #X_new : ndarray of shape (n_samples, n_components)
    #    Embedding of the training data in low-dimensional space.

```

```

$self->_check_params_vs_input($X);

# RESPALDO Quirúrgico para transformaciones futuras:
$self->{X_train_backup} = $X->copy();

my $embedding = $self->_fit($X);
return $self->{embedding_} = $embedding;
}

sub transform {
my ($self, $X_new) = @_;

die "El modelo no tiene las betas del entrenamiento original guardadas.\n"
    unless defined $self->{beta_train_};

my $X_train = $self->{X_train_backup};
my $n_train = $X_train->len;
my $n_new = $X_new->len;

# 1. Distancias cruzadas (Calculado eficientemente en MXNet)
my $X_new_sum = nd->sum($X_new * $X_new, axis => 1, keepdims => 1);
my $X_train_sum = nd->sum($X_train * $X_train, axis => 1, keepdims => 1);
my $cross_prod = nd->dot($X_new, $X_train->T);
my $sqdist_new = $X_new_sum - (2 * $cross_prod) + $X_train_sum->T;
$sqdist_new = nd->clip($sqdist_new, a_min => 0.0, a_max => 'Inf');

# 2. OPTIMIZACIÓN: Estimar Beta para los nuevos puntos sin búsqueda binaria
# openTSNE: Cada punto nuevo toma el promedio de las betas de sus K vecinos más cercanos de
# Para hacerlo puramente vectorial en MXNet:
my $k_neighbors = 5;
my $topk_k_indices = nd->topk($sqdist_new, k => $k_neighbors, axis => 1, is_ascend => 1);

# 2. Estimar Beta para los nuevos puntos sin búsqueda binaria
# Recolectar las betas correspondientes a esos índices y promediarlas
my $gathered_betas = nd->take($self->{beta_train_}->squeeze, $topk_k_indices);
my $precomputed_beta = nd->mean($gathered_betas, axis => 1, keepdims => 1);

# 3. Calcular P_new de forma directa (Pasamos la beta estimada, se salta los 100 pasos de b
my $P_new = $self->_binary_search_perplexity_mxnet($sqdist_new, undef, 0, $precomputed_beta);

# 4. Inicialización e Inyección en el optimizador parcial
my $init_new = nd->dot($P_new, $self->{embedding_});

my $X_combined = nd->concat($X_train, $X_new, dim => 0);
my $old_init = $self->{init};
$self->{init} = nd->concat($self->{embedding_}, $init_new, dim => 0);

# Optimización de coordenadas exclusivamente para las filas agregadas (skip_num_points)
my $embedded_combined = $self->_fit($X_combined, skip_num_points => $n_train);

$self->{init} = $old_init;
return $embedded_combined->slice([$n_train, $n_train + $n_new]);
}

1;
}

```

Out[6]: 1

```
Subroutine TSNE::mx redefined at /usr/local/share/perl5/5.30/AI/MXNet/NS.pm line 147.  
Subroutine new redefined at reply input line 13.  
Subroutine pdist redefined at reply input line 42.  
Subroutine squareform redefined at reply input line 98.  
Subroutine _binary_search_perplexity_scalar redefined at reply input line 165.  
Subroutine _binary_search_perplexity_mxnet redefined at reply input line 252.  
Subroutine _joint_probabilities redefined at reply input line 359.  
Subroutine _kl_divergence redefined at reply input line 374.  
Subroutine _gradient_descent redefined at reply input line 425.  
Subroutine _fit redefined at reply input line 493.  
Subroutine _check_params_vs_input redefined at reply input line 612.  
Subroutine fit_transform redefined at reply input line 634.  
Subroutine transform redefined at reply input line 662.
```

```
In [7]: use strict;  
use warnings;  
use Data::Dump qw(dump);  
use AI::MXNet qw(mx nd);  
use List::Util qw(max);  
use Time::HiRes qw(time);  
use sml qw(show_plot);  
IPerl->load_plugin('Chart::Plotly');
```

```
Subroutine main::mx redefined at /usr/local/share/perl5/5.30/AI/MXNet/NS.pm line 147.
```

```
In [8]: print "--- Ejecutando Experimento 1 ---\n";  
  
# 1. Datos de entrada idénticos  
my $X_data = nd->array([  
    [1.0, 0.1, -0.2, 0.5],  
    [1.1, 0.2, -0.1, 0.4],  
    [0.0, 5.2, 4.1, -3.0],  
    [0.1, 5.0, 4.3, -3.1],  
    [2.0, -1.0, 0.5, 1.2]  
], dtype => 'float64');
```

```
--- Ejecutando Experimento 1 ---
```

```
Out[8]: <AI::MXNet::NDArray 5x4 @cpu(0)>
```

```
In [9]: # 2. Calculamos las distancias condensadas con nuestra nueva pdist y squareform  
my $distances = TSNE->squareform(TSNE->pdist($X_data));  
nd->print($distances);
```

```
[[ 0.0000  0.0400 57.7500 58.0300  3.1900]  
 [ 0.0400  0.0000 55.4100 55.6500  3.2500]  
 [57.7500 55.4100  0.0000  0.1000 73.0400]  
 [58.0300 55.6500  0.1000  0.0000 72.5400]  
 [ 3.1900  3.2500 73.0400 72.5400  0.0000]]  
<AI::MXNet::NDArray 5x5 @cpu(0)> AI::MXNet::NDArray float64
```

```
Out[9]: 1
```

```
In [10]: # 3. Llamamos a las probabilidades conjuntas  
my $P = TSNE->_joint_probabilities($distances, 2.0, 0);
```

```
Out[10]: <AI::MXNet::NDArray 10 @cpu(0)>
```

```
In [11]: print "--- Vector condensado P ---\n";  
nd->printf("P:\n%.6f", $P);  
nd->printf("Suma total de P: %s\n", nd->sum($P));  
# --- Vector condensado P ---  
# P:  
# [0.117251 0.007424 0.007356 0.091662 0.008184 0.008123 0.090657 0.161504 0.003873 0.003968]
```

```
# <AI::MXNet::NDArray 10 @cpu(0)> AI::MXNet::NDArray float64
# Suma total de P: [0.5]
# <AI::MXNet::NDArray 1 @cpu(0)> AI::MXNet::NDArray float64
```

--- Vector condensado P ---

P:

```
[0.117251 0.007424 0.007356 0.091651 0.008184 0.008123 0.090667 0.161504 0.003873 0.003968]
```

```
<AI::MXNet::NDArray 10 @cpu(0)> AI::MXNet::NDArray float64
```

Suma total de P: [0.5]

```
<AI::MXNet::NDArray 1 @cpu(0)> AI::MXNet::NDArray float64
```

Out[11]: 1

```
In [12]: my $prob_matrix = TSNE->_binary_search_perplexity_mxnet($distances, 2, 0);
nd->printf("Probability Matrix:\n%.2f\n", $prob_matrix);
nd->printf("Sum of its columns:\n%.2f", nd->sum($prob_matrix, axis => 1));
```

Probability Matrix:

```
[[0.00 0.58 0.00 0.00 0.41]
```

```
 [0.59 0.00 0.00 0.00 0.41]
```

```
 [0.07 0.08 0.00 0.81 0.04]
```

```
 [0.07 0.08 0.81 0.00 0.04]
```

```
 [0.50 0.50 0.00 0.00 0.00]]
```

```
<AI::MXNet::NDArray 5x5 @cpu(0)> AI::MXNet::NDArray float64
```

Sum of its columns:

```
[1.00 1.00 1.00 1.00 1.00]
```

```
<AI::MXNet::NDArray 5 @cpu(0)> AI::MXNet::NDArray float64
```

Out[12]: 1

```
In [13]: print "Test de _kl_divergence\n";
my $p0_manual = nd->array([
    0.0001, -0.0002,
    -0.0001, 0.0003,
    0.0004, -0.0001,
    -0.0003, -0.0004,
    0.0000, 0.0000
], dtype => 'float64');

# Exageramos P para simular el estado de la iteración 0 de la Fase 1
my $P_exaggerated = $P * 12.0;
my ($cost, $grad_flat) = TSNE->_kl_divergence($p0_manual, [$P_exaggerated, 1, 5, 2], 0, 1);

print "--- Verificación Estática ---\n";
print "Costo inicial KL (Exagerado): $cost\n";
print "Gradiente Inicial (Primeros 4 elementos):\n";
nd->print($grad_flat->slice(end=>4));

# Test de _kl_divergence
# --- Verificación Estática ---
# Costo inicial KL (Exagerado): 37.5229988172705
# Gradiente Inicial (Primeros 4 elementos):
# [0.0015199281 -0.0035389844 -0.0015591937 0.0041695256]
```

Test de _kl_divergence

--- Verificación Estática ---

Costo inicial KL (Exagerado): 37.5229812584228

Gradiente Inicial (Primeros 4 elementos):

```
[ 0.0015 -0.0035 -0.0016 0.0042]
```

```
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray::Slice float64
```

Out[13]: 1

```
In [14]: print "--- Experimento 2: Trayectoria del optimizador ---\n";
# Este experimento confirmará si la acumulación iterativa del gradiente dentro del bucle mantie

# 1. Datos de entrada
$X_data = nd->array([
    [1.0, 0.1, -0.2, 0.5],
    [1.1, 0.2, -0.1, 0.4],
    [0.0, 5.2, 4.1, -3.0],
    [0.1, 5.0, 4.3, -3.1],
    [2.0, -1.0, 0.5, 1.2]
], dtype => 'float64');
```

```
$p0_manual = nd->array([
    0.0001, -0.0002,
    -0.0001, 0.0003,
    0.0004, -0.0001,
    -0.0003, -0.0004,
    0.0000, 0.0000
], dtype => 'float64');
```

--- Experimento 2: Trayectoria del optimizador ---

Out[14]: <AI::MXNet::NDArray 10 @cpu(0)>

```
In [15]: # 2. Calcular distancias y probabilidades conjuntas usando el flujo plano condensado
$distances = TSNE->squareform(TSNE->pdist($X_data));
nd->printf("distances:\n%.6f", $distances);
$P = TSNE->_joint_probabilities($distances, 2.0, 0);

$P_exaggerated = $P * 12.0;
```

```
# 3. Inicializar variables del optimizador en Perl
my $p = $p0_manual->copy();
my $update = nd->zeros($p->shape, dtype => 'float64');
my $gains = nd->ones($p->shape, dtype => 'float64');

my $learning_rate = 200.0;
my $momentum = 0.5;
my $min_gain = 0.01;

# distances:
# [[ 0.000000  0.040000 57.750000 58.030000  3.190000]
# [ 0.040000  0.000000 55.410000 55.650000  3.250000]
# [57.750000 55.410000  0.000000  0.100000 73.040000]
# [58.030000 55.650000  0.100000  0.000000 72.540000]
# [ 3.190000  3.250000 73.040000 72.540000  0.000000]]
# <AI::MXNet::NDArray 5x5 @cpu(0)> AI::MXNet::NDArray float64
```

```
distances:
[[ 0.000000  0.040000 57.750000 58.030000  3.190000]
 [ 0.040000  0.000000 55.410000 55.650000  3.250000]
 [57.750000 55.410000  0.000000  0.100000 73.040000]
 [58.030000 55.650000  0.100000  0.000000 72.540000]
 [ 3.190000  3.250000 73.040000 72.540000  0.000000]]
<AI::MXNet::NDArray 5x5 @cpu(0)> AI::MXNet::NDArray float64
```

Out[15]: 0.01

```
In [16]: for my $i (0 .. 49) {
    my ($cost, $grad) = TSNE->_kl_divergence($p, [$P_exaggerated, 1, 5, 2], 0, 1);

    if (($i + 1) % 25 == 0) {
        printf "Iteration %d: cost = %.6f, grad_norm = %.6f\n", ($i + 1), $cost, nd->norm($grad)->a
    }

    # Actualización adaptativa de ganancias vectorizada
    my $inc = nd->lesser_scalar(($update * $grad), 0.0);

    $gains = nd->where($inc, $gains + 0.2, $gains * 0.8); # 2. Ganancias adaptativas
    $gains = nd->clip($gains, a_min => $min_gain, a_max => 'Inf');
    $grad *= $gains; # 3. Actualizar gradiente
    $update = ($momentum * $update) - ($learning_rate * $grad); # 4. Actualizar Momentum
    $p += $update; # 5. Actualizar Posición
}

nd->print("\nCoordenadas p finales tras 50 iteraciones:\n", $p);

# Iteración 25: costo = 47.212835, grad_norm = 0.026793
# Iteración 50: costo = 59.882398, grad_norm = 0.158328

# Coordenadas p finales tras 50 iteraciones:
# [-131.9201  87.8838 -784.9028 -104.8960  75.5367 -120.0497  85.6921 -85.4233 114.
# <AI::MXNet::NDArray 10 @cpu(0)> AI::MXNet::NDArray float64
```

Iteration 25: cost = 43.550918, grad_norm = 0.011099
Iteration 50: cost = 43.518096, grad_norm = 0.016039

Coordenadas p finales tras 50 iteraciones:

```
[ 281.4920 -669.9366 -311.3163  825.5163  526.2187  223.8879 -544.3601 -229.5745   38.6100 -166.7373]
```

<AI::MXNet::NDArray 10 @cpu(0)> AI::MXNet::NDArray float64

Out[16]: 1

In [17]: `print "--- Tercero experimento: Test de _gradient_descent ---\n";`

```
# 1. Definir X_embedded (vector plano de 10 elementos)
my $X_embedded = nd->array([
    0.0001, -0.0002,
    -0.0001,  0.0003,
    0.0004, -0.0001,
    -0.0003, -0.0004,
    0.0000,  0.0000
], dtype => 'float64');

# 2. Vector P condensado de 10 elementos que obtuvimos del test pdist/squareform
$P = nd->array([
    0.11725088, 0.00742363, 0.00735645, 0.09166174,
    0.00818382, 0.00812266, 0.09065659, 0.16150376,
    0.00387255, 0.00396793
], dtype => 'float64');

$P *= 12; # early_exaggeration = 12
```

--- Tercero experimento: Test de _gradient_descent ---

Out[17]: <AI::MXNet::NDArray 10 @cpu(0)>

In [18]: `my ($n_samples, $n_components) = (5, 2);
my $degrees_of_freedom = max($n_components - 1, 1);
my $obj_func = \&{'TSNE::kl_divergence'};
my %opt_args = (it => 0,
 n_iter_check => 50,
 min_grad_norm => 1e-7,
 _learning_rate => 200,
 verbose => 2,
 skip_num_points => 0,
 objective_args => [$P, $degrees_of_freedom, $n_samples, $n_components],
 n_iter_without_progress => 250, # Limit of initial exploration
 n_iter => 250,
 momentum => 0.5,
);`

Out[18]: it0n_iter_check50min_grad_norm1e-07_learning_rate200verbose2skip_num_points0objective_argsARRAY (0x5609bcbbe800)n_iter_without_progress250n_iter250momentum0.5

In [19]: `my ($params, $kl_divergence, $it) = TSNE->_gradient_descent($obj_func, $X_embedded, %opt_args);
nd->printf("\nCoordenadas finales de P:\n", $params->reshape([$n_samples, $n_components]));`

```
# Coordenadas finales de P:
# [[ -38.2084 -108.8306]
# [ -73.0986 -119.3821]
# [-213.2087  148.8234]
# [ 119.5532   71.6138]
# [  41.7473 -107.3519]]
# <AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray float64
```



```
[t-SNE] Iteration 50 : error = 43.51745152, gradient norm = 0.11637446 (50 iterations in 0.057s)

[t-SNE] Iteration 100 : error = 53.72625499, gradient norm = 0.06015247 (50 iterations in 0.112s)

[t-SNE] Iteration 150 : error = 47.88393126, gradient norm = 1.46083829 (50 iterations in 0.167s)

[t-SNE] Iteration 200 : error = 48.21649153, gradient norm = 0.06930062 (50 iterations in 0.221s)

[t-SNE] Iteration 250 : error = 36.66340075, gradient norm = 0.39704566 (50 iterations in 0.276s)
```

Coordenadas finales de P:

```
[[ -38.2084 -108.8306]
 [ -73.0986 -119.3821]
 [-213.2087  148.8234]
 [ 119.5532   71.6138]
 [  41.7473 -107.3519]]
```

```
<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray float64
```

Out[19]: 1

```
In [20]: print "--- Cuarto experimento: fit_transform ---\n";
```

```
my $X = nd->array([
    [0.0, 1.0, 2.0],
    [3.0, 4.0, 5.0],
    [1.0, 0.0, 1.0],
    [9.0, 8.0, 7.0],
    [2.0, 2.0, 2.0],
], dtype => 'float64');

# Instanciar el objeto t-SNE

$p0_manual = nd->array([
    0.0001, -0.0002,
    -0.0001, 0.0003,
    0.0004, -0.0001,
    -0.0003, -0.0004,
    0.0000, 0.0000],
dtype => 'float64');
```

```
--- Cuarto experimento: fit_transform ---
```

```
Out[20]: <AI::MXNet::NDArray 10 @cpu(0)>
```

```
In [21]: my $tsne = new TSNE(
    n_components => 2,
    perplexity   => 2.0,
    init         => 'random', # $p0_manual, # Also init => 'random' with a random_state => 42
    random_state => 42,
    max_iter     => 250,
    learning_rate => 'auto',
    verbose      => 1 # 0, 1 or 2
);
```

```
Out[21]: TSNE=HASH(0x5609b6f4cc40)
```

```
In [22]: $X_embedded = $tsne->fit_transform($X);
```

```
[t-SNE] Computing pairwise distances...
[t-SNE] Starting Early Exaggeration Phase...
[t-SNE] KL divergence after 150 iterations with early exaggeration: 35.55358770 in 0.275s
[t-SNE] KL divergence after 250 iterations: 0.04240231 in 0.384s
```

```
Out[22]: <AI::MXNet::NDArray 5x2 @cpu(0)>
```

```
[t-SNE] Warning: n_samples (5) es menor que 3 * perplexity. El resultado podría ser inestable.
```

```
In [23]: nd->printf("Coordenadas finales desde fit_transform:\n%.4f", $X_embedded);
```

```

Coordenadas finales desde fit_transform:
[[ -10.8137  -43.6784]
 [  18.2323  162.6158]
 [ -96.1024  -89.1358]
 [ -59.6290  264.7139]
 [ -66.4577   46.5643]]
<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray float64

```

Out[23]: 1

```

In [24]: print "--- Quinto experimento: Comprobacion de resultados ---\n";

# Comprobacion del resultado mediante Invarianza a la rotación y traslación: Si tomas el mapa d
# lo rotas 90 grados y lo desplazas en el espacio, las distancias relativas siguen siendo las m
# Aunque dos implementaciones de algoritmo t-SNE arrojen números visualmente distintos el uno
# si la matriz de distancias par a par conserva la misma proporción armónica,
# ambos mapas representan exactamente la misma solución bajo transformaciones afines.

sub calcular_distancias_par_a_par {
  my $X = shift; # Forma esperada: [n, dimensiones]

  # 1. Expandimos X a [n, 1, dimensiones] y a [1, n, dimensiones]
  my $X_expand_i = $X->expand_dims(axis => 1);
  my $X_expand_j = $X->expand_dims(axis => 0);

  # 2. El broadcasting resta automáticamente todas las combinaciones: [n, n, dimensiones]
  my $diff_matrix = $X_expand_i - $X_expand_j;

  # 3. Elevamos al cuadrado, sumamos en el eje de las dimensiones y sacamos raíz cuadrada
  # Nota: Ajusta el nombre de los métodos según tu librería exacta (ej: square, sum, sqrt)
  my $dist_matrix = $diff_matrix->square()->sum(axis => -1)->sqrt();

  return $dist_matrix; # Forma final: [n, n]
}

```

--- Quinto experimento: Comprobacion de resultados ---

Out[24]: 1

```

In [25]: # 2. Definir matriz de rotación de 90 grados (en radianes: pi/2)
# R = [[cos(theta), -sin(theta)], [sin(theta), cos(theta)]] -> [[0, -1], [1, 0]]
my $R = nd->array([
  [0.0, -1.0],
  [1.0, 0.0]
], dtype => 'float64');

# 2. Aplicar rotación y una traslación arbitraria (ej. +500 en X, -300 en Y)
my $X_rotado = nd->dot($X_embedded, $R->T);
my $X_transformado = $X_rotado + nd->array([500.0, -300.0], dtype => 'float64');

my $dist_orig = calcular_distancias_par_a_par($X_embedded);
my $dist_tran = calcular_distancias_par_a_par($X_transformado);

nd->printf("--- TRANSFORMED COORDINATES ---\n%.4f", $X_transformado);
printf "Maximum absolute difference in relative distances: %s\n",
  nd->max($dist_orig - $dist_tran)->abs->asscalar;
printf "Is the transformed map geometrically equivalent?: %d\n",
  nd->_contrib_allclose($dist_orig, $dist_tran)->asscalar;

```

--- TRANSFORMED COORDINATES ---

```

[[ 543.6784 -310.8137]
 [ 337.3842 -281.7677]
 [ 589.1358 -396.1024]
 [ 235.2861 -359.6290]
 [ 453.4357 -366.4577]]

```

```

<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray float64
Maximum absolute difference in relative distances: 5.6843418860808e-14
Is the transformed map geometrically equivalent?: 1

```

Out[25]: 1

```

In [26]: print "--- Sexto experimento: Full Iris Dataset ---\n";

my ($dataset, $header) = sml->load_csv('/home/jehosha/Downloads/data/iris.csv');
my ($lookup, $rlookup) = sml->str_column_to_int($dataset, -1);
$dataset = nd->array($dataset);

```

```

# Extraer columnas 0 y 2 para trabajar en un entorno 2D real
$X = $dataset->slice(':', [0, -1]);
my $y = $dataset->slice(':', -1);

# Standardize
my $means = nd->mean($X, axis=>0);
my $stdevs = nd->std($X, axis=>0, ddof=>1);
sml->standardize_dataset($X, $means, $stdevs);

print $X->slice([0, 5])>asstr;

$tsne = new TSNE(
    n_components => 2,
    perplexity => 15,
    init => 'random',
    random_state => 4,
    max_iter => 250,
    learning_rate => 'auto',
    verbose => 2, # 0, 1 or 2
    n_iter_check => 50,
);

$X_embedded = $tsne->fit_transform($X);
nd->print($X_embedded->slice([0, 5]));

# Plotting the result of the PCA data transformation

my $color_scale = [
    [0, 'green'],
    [0.5, 'purple'],
    [1, 'orange']
];

my $trace = new Chart::Plotly::Trace::Scatter(
    x => $X_embedded->slice(':', 0)->asarray,
    y => $X_embedded->slice(':', 1)->asarray,
    mode => 'markers',
    marker => {
        color => $y->asarray,
        colorscale => $color_scale,
        cmin => $y->min,
        cmax => $y->max,
        size => 10
    }
);

my $layout = {title => { text => 't-SNE' },
    axis => { title => 'Component 1 t-SNE' },
    yaxi => { title => 'Component 2 t-SNE' },
    width => 600, height => 550,
    margin => { l => 50, r => 0, t => 50, b => 50 }};

my $plot = new Chart::Plotly::Plot(traces => [$trace],
    layout => $layout);

IPerl->display($plot);

```

```

--- Sexto experimento: Full Iris Dataset ---
[[-0.8977  1.0286 -1.3368 -1.3086]
 [-1.1392 -0.1245 -1.3368 -1.3086]
 [-1.3807  0.3367 -1.3935 -1.3086]
 [-1.5015  0.1061 -1.2801 -1.3086]
 [-1.0184  1.2592 -1.3368 -1.3086]]
[t-SNE] Computing pairwise distances...
[t-SNE] Starting Early Exaggeration Phase...
[t-SNE] Iteration 50 : error = 54.26422309, gradient norm = 0.38250320 (50 iterations in 0.078s)

[t-SNE] Iteration 100 : error = 51.39255305, gradient norm = 0.38743009 (50 iterations in 0.158s)

[t-SNE] Iteration 150 : error = 50.57752759, gradient norm = 0.44137613 (50 iterations in 0.237s)

[t-SNE] Iteration 200 : error = 51.99420279, gradient norm = 0.43999508 (50 iterations in 0.316s)

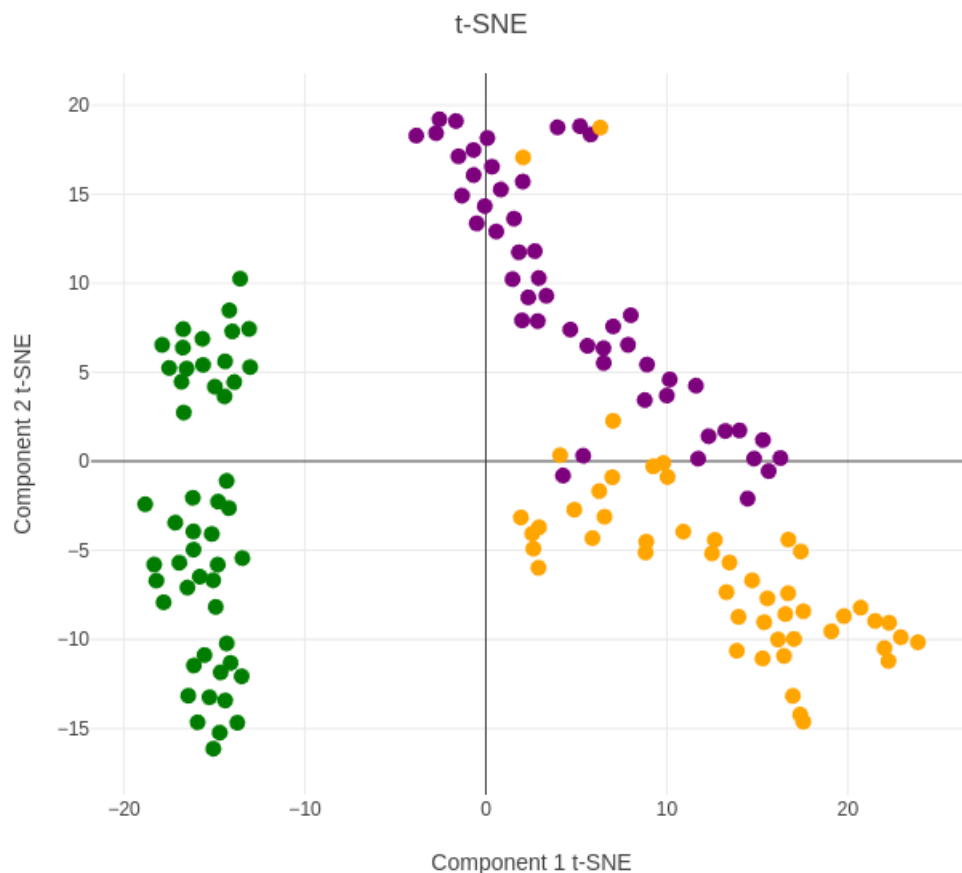
[t-SNE] Iteration 250 : error = 51.34627897, gradient norm = 0.37262739 (50 iterations in 0.394s)

[t-SNE] KL divergence after 150 iterations with early exaggeration: 50.57752759 in 0.395s
[t-SNE] Iteration 200 : error = 0.32118701, gradient norm = 0.00578632 (50 iterations in 0.077s)

[t-SNE] Iteration 250 : error = 0.27859919, gradient norm = 0.00374605 (50 iterations in 0.153s)

[t-SNE] KL divergence after 250 iterations: 0.27859919 in 0.548s
[[-19.8023  6.8934]
 [-15.2278 -1.1067]
 [-15.2284  1.4796]
 [-14.3511  0.5988]
 [-20.8983  7.2159]]
<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray::Slice float64

```



```

In [27]: # 2. Definir matriz de rotación de 90 grados (en radianes: pi/2)
# R = [[cos(theta), -sin(theta)], [sin(theta), cos(theta)]] -> [[0, -1], [1, 0]]
$R = nd->array([
    [0.0, -1.0],
    [1.0, 0.0]
], dtype => 'float64');

```

```
# 2. Aplicar rotación y una traslación arbitraria (ej. +500 en X, -300 en Y)
$X_rotado = nd->dot($X_embedded, $R->T);
$X_transformado = $X_rotado + nd->array([500.0, -300.0], dtype => 'float64');

$dist_orig = calcular_distancias_par_a_par($X_embedded);
$dist_tran = calcular_distancias_par_a_par($X_transformado);

nd->printf("--- TRANSFORMED COORDINATES ---\n%.4f", $X_transformado->slice([0, 5]));
printf "Maximum absolute difference in relative distances: %s\n",
      nd->max($dist_orig - $dist_tran)->abs->asscalar;
printf "Is the transformed map geometrically equivalent?: %d\n",
      nd->_contrib_allclose($dist_orig, $dist_tran)->asscalar;
```

```
--- TRANSFORMED COORDINATES ---
[[ 493.1066 -319.8023]
 [ 501.1067 -315.2278]
 [ 498.5204 -315.2284]
 [ 499.4012 -314.3511]
 [ 492.7841 -320.8983]]
<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray::Slice float64
Maximum absolute difference in relative distances: 6.92779167366098e-14
Is the transformed map geometrically equivalent?: 1
```

Out[27]: 1

```
In [28]: print "--- Séptimo experimento: Muestreo Tensorial y Proyección de Nuevos Puntos ---\n";

# 1. El dataset está agrupado consecutivamente (50 muestras por clase)
# $X tiene dimensiones [150, 4] y $y tiene [150, 1] (las etiquetas 0, 1, 2) ordenadas
my $class_means = $X->reshape([3, 50, 4])->mean(axis => 1);

# 2. Agregar ruido aleatorio normal a las 4 dimensiones para simular 3 puntos aleatorios válido
my $noise = nd->random->normal(loc => 0.0, scale => 0.2, shape => [3, 4]);
my $X_new_3d = $class_means + $noise;

nd->printf("Nuevos 3 puntos generados en 4D (uno por clase):\n%.4f\n", $X_new_3d);

# 3. Proyectar usando el método transform ultra-optimizado que guarda betas
$tsne->{verbose} = 0;
my $X_new_embedded = $tsne->transform($X_new_3d);
nd->printf("Coordenadas 2D proyectadas para los nuevos puntos:\n%.4f\n", $X_new_embedded);
```

```
--- Séptimo experimento: Muestreo Tensorial y Proyección de Nuevos Puntos ---
Nuevos 3 puntos generados en 4D (uno por clase):
[[-1.0607  0.9483 -1.1132 -1.1029]
 [ 0.0461 -0.6381  0.2797  0.1059]
 [ 1.2415 -0.2400  1.2199  1.3044]]
<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray float32

Coordenadas 2D proyectadas para los nuevos puntos:
[[-19.2119  5.8176]
 [ 2.5474  1.7182]
 [ 15.4023 -7.4926]]
<AI::MXNet::NDArray 3x2 @cpu(0)> AI::MXNet::NDArray::Slice float64
```

Out[28]: 1

```
In [29]: # =====
# Visualización con Chart::Plotly (Marcando los nuevos puntos con una X)
# =====

# Reutilizamos el $trace original de los 150 puntos del Iris dataset

my $trace_new_points = new Chart::Plotly::Trace::Scatter(
  x    => $X_new_embedded->slice(':', 0)->asarray,
  y    => $X_new_embedded->slice(':', 1)->asarray,
  mode => 'markers',
  name => 'New points',
  marker => {
    symbol => 'cross',
    color  => 'red',
    line   => { width => 3, color => 'black' },
    size   => 16,
  }
);
```

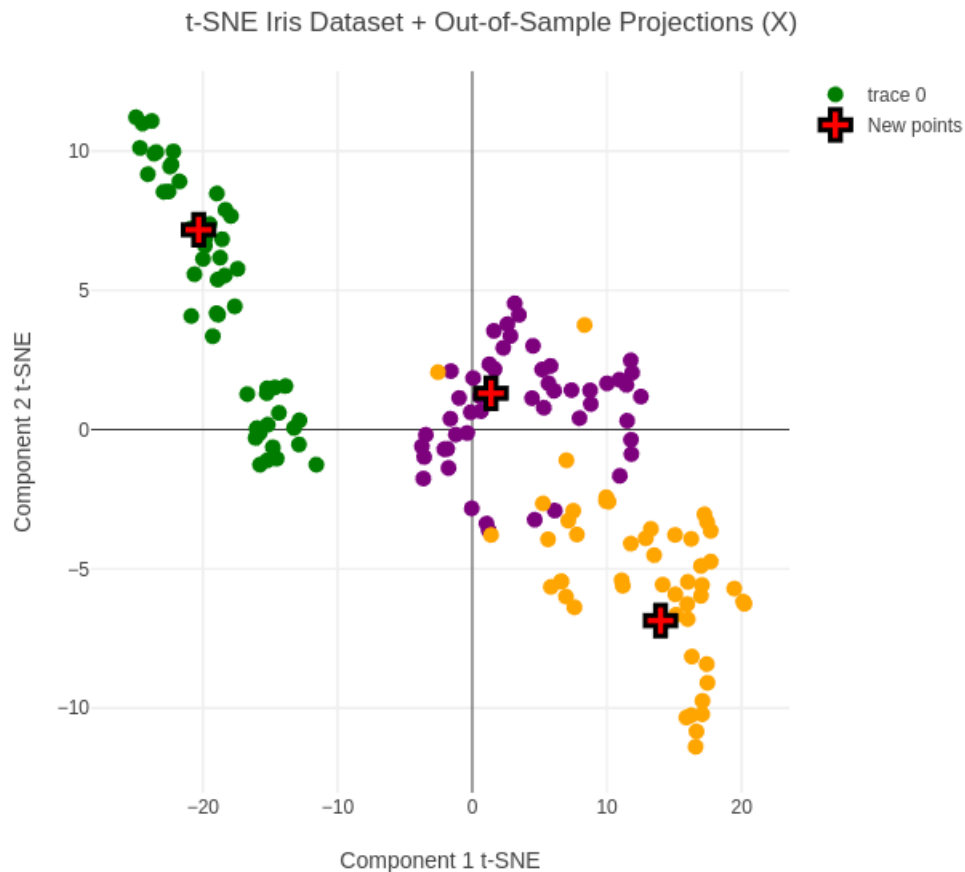
```

$layout = {title => { text => 't-SNE Iris Dataset + Out-of-Sample Projections (X)' },
  xaxis => { title => 'Component 1 t-SNE' },
  yaxis => { title => 'Component 2 t-SNE' },
  width => 600, height => 550,
  margin => { l => 50, r => 0, t => 50, b => 50 }};

$plot = new Chart::Plotly::Plot(traces => [$trace, $trace_new_points],
  layout => $layout);

IPerl->display($plot);

```



Parametrización y pasos siguientes

Con respecto a los hiperparámetros del algoritmo t-SNE, se usa una tasa de aprendizaje (learning rate) inusualmente grande (comúnmente entre 100 y 1000) porque las fuerzas de atracción entre grupos de datos se debilitan rápidamente a medida que se alejan durante el proceso de reducción de dimensiones.

¿Por qué necesita un valor tan alto?

- Fuerza de atracción: Imagina que el algoritmo es como un imán para agrupar datos similares. Si el imán está débil (tasa de aprendizaje baja), los grupos tardarán una eternidad en juntarse o se quedarán estancados.
- Evitar atascos: Un valor alto ayuda al algoritmo a dar "pasos grandes", lo que le permite escapar de pequeños errores (mínimos locales) y encontrar la mejor posición general para los datos.

El efecto de la tasa de aprendizaje

- Muy alta: Los puntos de datos se empujan demasiado y terminan formando un círculo o "bola" sin mucho sentido.
- Muy baja: Los puntos se agrupan en una nube muy apretada y densa, dificultando ver los detalles.

Para el conjunto de datos Iris (150 filas y 3 clases), los parámetros óptimos para t-SNE son una perplejidad (perplexity) entre 10 y 30 y una tasa de aprendizaje (learning rate) de 200 o 'auto'.

Al ser un dataset muy pequeño, valores estándar demasiado altos dispersarán por completo los grupos.

Configuración Recomendada:

- `perplexity = 15`

La perplejidad balancea la atención entre la estructura local y global. Regla general: Debe ser menor que el número de puntos. Para 150 filas, un valor entre 10 y 30 evita que los 3 clústeres se distorsionen.

- `learning_rate = 200` (o usar 'auto')

En datasets pequeños, una tasa de 1000 es excesiva. El valor predeterminado 'auto' calcula automáticamente $(\max(200, N/12/4))$, lo que para Iris resulta exactamente en 200.

- `n_iter = 250`

Número de iteraciones suficientes para que el mapa converja y los 3 grupos se estabilicen visualmente.

- `random_state = semilla`

Para evaluar la calidad de la separación de las etiquetas, debe realizar experimentos variando la para encontrar una que permita separar los clusters acorde a las etiquetas del dataset.

A partir del resultado de t-SNE se usa un algoritmo de clustering como K-Means o GMM para asignar etiquetas formales a los grupos visuales descubiertos, ya que t-SNE solo proyecta los datos en dos o tres dimensiones para su visualización, pero no identifica ni delimita numéricamente a qué clúster pertenece cada punto.

¿Por qué se combinan ambos métodos?

- **t-SNE visualiza:** Agrupa los puntos geoméricamente en la pantalla basándose en su similitud inicial, pero no genera un modelo predictivo ni guarda fronteras.
- **Clustering automatiza:** Algoritmos como K-Means o GMM (Gaussian Mixture Models) toman esas coordenadas simplificadas y calculan matemáticamente los límites de cada grupo para clasificar automáticamente los datos nuevos o existentes.

In []: